

Hardening Roadmap

Known gaps between this platform and one that could hold real money

Trading Platform

Purpose

This platform is a paper-trading system built to exercise real exchange plumbing: an in-memory risk engine on the order hot path, a write-ahead log, and event-driven persistence. It is deliberately **not** production-hardened.

Rather than leave that implicit, this document names each gap, explains the failure it would cause, says why the current design does not cover it, and sketches the fix. Everything here is a considered deferral, not an oversight.

The distinction that governed what got built versus deferred: **correctness bugs were fixed, resilience features were documented.** A lost-update race that silently corrupts a balance is wrong at any scale, so it was fixed. A single Kafka broker is not wrong, it is simply not highly available — and building for availability that nothing needs is how a portfolio project turns into fourteen half-finished features.

Durability and recovery

Engine crash between the state mutation and the WAL flush

Failure. `execute_order` mutates the in-memory balance and then calls `write_fill`, both under the user's lock. If the process dies between those two statements, the engine's memory said the order filled but nothing was ever written. On restart the engine reloads from Postgres, which never heard about it, so the fill vanishes. The caller may already have received `201 FILLED`.

Why the current design does not cover it. The WAL is written *after* the mutation, not before. A true write-ahead log writes the intent first, so that a crash leaves a record to replay.

Fix. Invert the order inside the lock: write an intent record, flush it, mutate, then write a commit marker. On startup, scan the tail of `wal.log` for intent records with no matching commit and either replay or discard them. `event_id` already exists to pair the two. The cost is a second `fsync` per fill, which for this workload is acceptable.

WAL disk-full or partial write

Failure. `write_fill` does not check whether the write succeeded. A full disk means `file << ...` silently fails, the in-memory state moves on, and the fill never reaches Postgres.

Why not covered. `WALWriter` never inspects the stream state.

Fix. Check `file.good()` after the flush and, on failure, fail the RPC with `INTERNAL_ERROR` and refuse to mutate state. That makes a full disk a clean rejection instead of a silent divergence. Add a disk-space check at startup.

Engine restart loses the idempotency cache

Failure. The idempotency cache lives in `userState`, in memory. After a restart, a client retrying with the same `client_order_id` gets executed a second time — the engine has no memory of the first.

Why not covered. The cache is deliberately in-process; that is what makes it fast enough to sit on the hot path.

Partially mitigated. `fill_consumer` checks `orders.idempotency_key` and the column carries a unique constraint, so the duplicate is rejected before it reaches Postgres. The gap is that the *caller* still receives a second `FILLED` response for an order that will never be persisted.

Fix. Have the engine consult Postgres for the key on a cache miss, or rebuild the cache on startup from recent orders rows. The second is cheaper and bounded: load the last N minutes of keys per active account.

Delivery guarantees

Kafka exactly-once

Failure. `wal_producer` sends to Kafka, then saves its file cursor. A crash between those two steps replays the last line on restart, so the same fill is delivered twice.

Why not covered. The producer is a plain tail-and-send loop with an at-least-once guarantee.

Mitigated, not solved. `fill_consumer` is idempotent — it skips a fill whose `order_id` or `client_order_id` is already applied, and the unique constraint catches a race past both checks. So duplicates are harmless today. What is not handled is the opposite case: a crash *before* the send, where the cursor was already advanced, would lose a fill entirely. The current ordering makes that impossible, at the cost of allowing duplicates — the right trade, but worth naming.

Fix. Kafka transactions (`enable.idempotence`, transactional producer) with the cursor committed inside the transaction.

Per-user ordering across multiple partitions

Failure. `fill_consumer` writes the *absolute* new balance carried in each event, not a delta. If two fills for one user are consumed out of order, the final balance is the older of the two — silently wrong.

Why not fully covered. `wal_producer` now keys messages by `user_id`, so a user's fills hash to one partition and Kafka guarantees order within it. That closes the common case. Two things remain:

- **Repartitioning.** The partition is `hash(key) % partition_count`. Increasing the partition count remaps users, so a user's older messages sit on the old partition while new ones go elsewhere — ordering can break exactly once, at the boundary.
- **Nothing verifies it.** `account_sequence` is written into every event specifically so a consumer can detect a gap or a regression, but no consumer acts on it.

Fix. Have `fill_consumer` track the last applied `account_sequence` per account and refuse to apply anything that is not exactly one greater, parking out-of-order messages until the gap fills. For repartitioning, drain the topic first or over-provision partitions up front.

Single Kafka broker

Failure. The broker is a single point of failure; if it dies, fills stop reaching Postgres. `wal.log` keeps growing, so nothing is lost, but the database falls arbitrarily far behind.

Why not covered. `docker-compose.yml` runs one broker with `KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1`. Deliberate — a three-broker cluster on a laptop buys nothing for a demo.

Fix. Three brokers, `replication.factor=3`, `min.insync.replicas=2`, and a `producer.acks=all`.

Consumer throughput

Failure. `fill_consumer` commits one message at a time in a single process. Under real volume across 300+ symbols, consumer lag grows and Postgres falls behind.

Why not covered. Deliberate — the order hot path is already async, so this only affects how quickly the database catches up. The right response is to measure before optimising.

Fix, when measurement justifies it. Batch commits every N messages, or run multiple consumers in a group with the topic partitioned. The `user_id` key already makes the second safe.

Correctness at the edges

The engine works in doubles

Failure. `UserStateStore` holds `cash` and `quantity` as `double`. Repeated addition and subtraction accumulates representation error, and the funds check `user.cash < cost` compares approximate values. Postgres stores exact `numeric(28,8)`, so the two can drift.

Why not covered. Doubles are what keeps the hot path fast, and the persistence boundary quantises to 8 decimal places (`core/money.py`), so the stored values are exact and consistent. The drift is bounded and, for paper trading, invisible.

Fix. Represent money in the engine as a 64-bit integer count of the smallest unit ($1e-8$), the way real matching engines do. Arithmetic becomes exact and the comparison becomes trivially correct.

Audit-trail columns are unpopulated

Failure. `risk_checks`, `orders.rejection_reason`, `orders.requested_price` and `positions.realised_pnl` exist in the schema but nothing writes to them. There is no record in Postgres of a declined order or why it was declined, because only fills flow through the WAL.

Why not covered. Only `fill_consumer` writes Order rows, and it only ever sees fills.

Fix. `ExecuteOrder` already returns a typed rejection with the real shortfall figures. Persist rejections from `orderService` directly (they need no WAL round-trip, since nothing was mutated), setting `status`, `rejection_reason` and `requested_price`, and write a `RiskCheck` row per decision. Realised P&L needs the sell side of `execute_order` to compute it against `average_price` and put it in the WAL.

No risk limits beyond solvency

Failure. The engine checks only “can they afford it”. There is no per-account exposure cap, no leverage limit, and no kill switch. A runaway strategy can trade its entire balance in a loop.

Why not covered. Solvency is the only limit a paper-trading demo needs.

Fix. Per-account maximum notional and maximum position size checked inside `execute_order` under the same lock, plus a global halt flag the engine reads before executing. All three are cheap because the lock is already held.

Fills always hit top of book

Failure. Order size is ignored. Buying 1,000 BTC fills entirely at the best ask, which in reality would walk several levels and cost significantly more. No slippage, no fees, no partial fills, no latency.

Why not covered. The order book stores full depth (bids/asks vectors) but `execute_order` only reads `best_bid/best_ask`.

Fix. Walk the levels, accumulating quantity until the order is filled, returning the volume-weighted average price — and a partial fill if depth runs out. The data is already there; this is arithmetic, not plumbing.

Operational

No retry or circuit breaker around gRPC

Failure. If the engine is briefly unavailable, every order fails immediately with 503. A restart drops all in-flight requests.

Why not covered. `RiskEngineClient` catches `grpc.RpcError` and raises `RiskEngineUnavailableError` with no retry.

Fix. Retry with exponential backoff for the idempotent case — which is now safe precisely *because* `client_order_id` makes a retried `ExecuteOrder` harmless. Add a circuit breaker so a downed engine is not hammered.

Observability is one metric

Failure. `/metrics` exposes only `execute_order_latency_seconds`. There is no visibility into fill throughput, consumer lag, WAL growth, rejection rates by reason, or feed connection health.

Fix. Counters for orders by result code (the typed enum makes this free), consumer lag as a gauge, WAL file size, and per-shard feed connection state. Then Prometheus scraping and a Grafana dashboard.

Dashboard reads CSV files

Failure. The Streamlit dashboard polls CSVs written by `strategy_runner`. It can read a file mid-write, the files grow without bound, and supporting multiple strategies or dashboard instances is awkward.

Why not covered. Deliberate: it keeps the dashboard fully decoupled from the trading path, so a dashboard query can never block an order or hold a database lock. For one strategy and one viewer, files are adequate.

Fix. A `strategy-state` Kafka topic and a consumer maintaining a materialised view in Redis, which the dashboard reads. Preserves the decoupling while adding ordering, replay and multi-strategy support.

No strategy plugin lifecycle

Failure. One hardcoded `ThresholdStrategy` runs in one process. There is no way to register strategy code at runtime, run several concurrently with separate books, or start and stop one without restarting the runner.

Fix. The strategy base class in `strategies/base.py` is already the right seam. What is missing is a registry, a per-strategy account, and supervision.

Fixed since first draft. `ThresholdStrategy.on_price` used to flip `self.holding[symbol]` before the order was known to have succeeded, so a rejected order left the strategy believing it held a position it did not — and since it only sells what it thinks it holds, that symbol stopped trading. The strategy contract now separates deciding from committing: `on_price` is pure, and `on_fill`

/ on_reject report the real outcome back. A side benefit is that the reference price now anchors to the actual execution price rather than the mid-price the decision was made on.

Testing gaps

The suite is 68 tests: engine-level concurrency and idempotency against a real `risk_engine.exe`, account isolation against a real database, request validation, the result-code mapping, and the accounting rules.

Not covered:

- **Full-stack end-to-end.** Nothing drives `Flask` → engine → WAL → Kafka → `fill_consumer` → Postgres in one automated test. It has been verified by hand but is not pinned down.
- **Killing the consumer mid-fill** and asserting the fill is applied exactly once on restart.
- **Concurrent orders across different users**, which should be fully parallel since each account has its own lock, but nothing asserts it.
- **Feed reconnection** after a dropped Binance WebSocket shard.
- **Load.** `locust` is a dependency but there are no scenarios; consumer lag under sustained volume is unmeasured, which is exactly what the throughput question above needs before it can be answered.